

数据结构课程设计

指 导 书

东华大学计算机科学与技术学院

2026 年 6 月

目 录

1. 前言.....	1
1.1 指导思想.....	1
1.2 设计任务.....	1
1.3 参考进度.....	2
1.4 成绩评定.....	2
1.4.1 个人任务评分标准.....	2
1.4.2 小组任务评分标准.....	3
1.5 注意事项.....	4
1.6 参考书目.....	4
2. 个人任务.....	6
2.1 顺序表元素的快速删除.....	6
2.2 操作系统任务调度问题.....	6
2.3 判断单链表的对称性.....	6
2.4 AVL 树的判断.....	7
2.5 判断给定的二叉树是否是二叉排序树.....	7
2.6 希尔排序.....	7
2.7 堆排序.....	8
2.8 找出从指定结点出发且长度为 m 的所有简单路径.....	8
2.9 获取 AOE 网的关键路径.....	9
2.10 油管铺设.....	9
2.11 快速排序的应用.....	9
2.12 仓储货物重排.....	10
2.13 智能物流中心任务调度.....	11
2.14 大数据分析平台热点数据缓存.....	12
2.15 智能仓储分拣机器人调度.....	14
3. 小组任务.....	15
3.1. 任务背景.....	15
3.2. 基本功能.....	15
3.3. 主要功能模块.....	16
3.4. CSV 数据文件说明.....	17
3.5. 部分项目代码结构（建议）.....	18
3.6. 小组成员分工建议（4 人为例）.....	19
3.7. 实验报告的要求.....	19
3.8. 运行效果图（部分）.....	20
附录 上海地铁网络示意图.....	25

1. 前言

《数据结构》是计算机科学与技术专业的一门核心专业基础课程，其主要任务是培养学生的算法设计能力及良好的程序设计习惯。通过学习，要求学生掌握典型算法的设计思想及程序实现，能够根据实际问题选取合适的存储方案、设计出简洁、高效、实用的算法，并为后续课程的学习及软件开发打下良好的基础。

1.1 指导思想

本次课程设计的指导思想是：

- 1、 学习获取知识的方法；
- 2、 提高发现问题、分析问题和解决实际问题的能力；
- 3、 加强创新意识和创新精神；
- 4、 加强团队的分工与合作；
- 5、 掌握面向实际背景思考问题的方法。

1.2 设计任务

本次课程设计任务主要分为个人任务和小组任务两部分。

个人基本任务：

在 DHU-OJ 平台上按要求完成“个人任务”部分的设计任务。

小组任务：

使用通义灵码完成上海地铁线路查询系统。

要求：

- 1、 每项目小组人员为 3 - 5 名。
- 2、 每项目小组提交一份课程设计报告，每人提交一份个人任务的设计报告，具体内容见实验报告模板。

1.3 参考进度

课程设计总共包括 10 大节课，参考进度如下：

第 1 节，布置任务，了解任务要求，分小组。

第 1-6 节，完成个人任务

第 7-9 节，完成小组任务、检查个人任务。

第 10 节，检查小组任务。

1.4 成绩评定

采用小组考核和个人考核两级考核方法（各占 50%）。

考核项	个人任务（50%）	小组任务（50%）	占总成绩比例（%）
上机代码	60	60	60
实验报告	40	40	40
占总成绩比例（%）	100	100	100

1.4.1 个人任务评分标准

个人任务满分 100 分，其中程序设计与实现 60 分，实验报告 40 分。

程序设计与实现部分主要考察数据结构选择、算法设计、功能实现、程序正确性以及代码规范性，提交的程序应能够正确运行并通过测试数据验证。

实验报告部分主要考察问题分析、数据结构设计说明、算法设计说明、系统实现说明、测试结果分析以及课程设计总结。实验报告应完整反映设计过程和算法实现思路。对于功能缺失、算法错误、测试分析不足或报告内容不完整的情况，将按照评分标准酌情扣分。

上机代码（60 分）		
数据结构设计	数据结构选择合理，能够体现题目对应知识点，设计规范	10
算法设计	算法思路正确，复杂度合理，能够体现课程知识点应用	10
功能完整性	完成题目要求的全部功能，无功能缺失	15
程序正确性	测试结果正确，边界情况处理合理，无明显逻辑错误	15

代码规范性	命名规范、格式统一、注释清晰、结构合理	10
实验报告（40 分）		
问题分析	对题目背景、业务场景、需求进行分析，明确输入输出要求	5
数据结构设计说明	说明所采用的数据结构及选择理由	8
算法设计说明	给出算法思想、流程描述或伪代码，并分析时间复杂度和空间复杂度	8
系统实现说明	对主要模块和关键代码进行说明	5
测试结果分析	不少于 5 组测试数据，包含边界测试，并对结果进行分析	8
总结与心得	总结设计过程中的收获、存在的问题及改进方向	6

1.4.2 小组任务评分标准

小组任务满分 100 分，其中程序设计与实现 60 分，实验报告 30 分，答辩与演示 10 分。

上机代码（60 分）		
CSV 数据集建设	完成 Station.csv、Edge.csv 等数据集设计与构建，数据完整合理	10
图结构构建	正确建立地铁网络图，支持换乘站建模	10
路径规划功能	实现最短时间路径、最少换乘路径等核心功能	15
运营管理功能	实现站点关闭、线路停运、换乘站关闭、线路查询等功能	15
网络分析功能	实现受影响区域分析等功能	5
代码规范与系统稳定性	结构清晰、模块化设计、运行稳定	5
实验报告（30 分）		
需求分析	问题背景、功能需求、业务流程分析完整	5
总体设计	系统架构、模块划分、数据结构设计合理	5
详细设计	核心算法、关键模块设计说明完整	8
测试与结果分析	测试数据充分，结果分析合理	7
总结与展望	总结完整，能够提出改进方向	5
答辩与演示（10 分）		
系统演示	功能展示完整、操作熟练	5
问题回答	能准确说明数据结构、算法及系统设计思路	5

小组任务的成绩折算到个人成绩的方法：

小组课程设计报告封面包含一份成员贡献表，表格式如下所示。

例：下表中，如果成员数为 4，则贡献度总和为 4。

如果小组成绩为 80 分，则折算到个人的成绩如下：

张三：80*1.1=88 分

李四：80*1=80 分

王五：80*0.95=76 分

钱六：80*0.95=76 分

学号	姓名	贡献度
101	张三	1.1
102	李四	1
103	王五	0.95
104	钱六	0.95
	(总计)	4

如果某小组无此表格，则每个成员的贡献度按 1 计算。

如果某小组的贡献度平均值大于 1，则降低组长的贡献度，使得平均值为 1。

1.5 注意事项

- 1、迟到 3 次或缺席一次，成绩下降一个档次，迟到 6 次或缺席 2 次，成绩再下降一个档次，依次类推。
- 2、课程设计开始前，各班的同学在班内自由组合，形成小组，每小组自行推荐小组长一人，在课程设计开始时上交组长名单、小组成员名单，名单上注明学号、姓名。

1.6 参考书目

- 1、严蔚敏等著，数据结构（C 语言版），人民邮电出版社
- 2、翁惠玉等著，数据结构：题解与拓展

3、余腊生等著，数据结构：基于 C++模版类的实现

4、菜鸟教程：C++ STL 教程

<https://www.runoob.com/cplusplus/cpp-stl-tutorial.html>

2. 个人任务

2.1 顺序表元素的快速删除

已知线性表 A 采用顺序存储结构，请设计一时间复杂度为 $O(n)$ 、空间复杂度为 $O(1)$ 的算法，该算法删除线性表中所有值为 `item` 的数据元素。要求：线性表元素个数 n 很大，而值为 `item` 的数据元素个数很少，**要求移动元素个数尽量少**；删除后的数组元素与原数组元素不必保持顺序一致。

提示：算法参照快速排序的思想，使用两个指针，分别从左往右扫描找到需要被删除的元素、从右往左扫描找到不被删除的元素，然后交换。然后继续扫描。

参考函数原型：

```
//删除线性表中所有值为 item 的数据元素，时间复杂度为  $O(n)$ 、空间复杂度为  $O(1)$ 
template<class ElemType>
void DeleteItem( vector<ElemType> &A, ElemType item );
```

2.2 操作系统任务调度问题

设计并实现一个算法 `Schedule`，完成简单的操作系统任务调度问题。

操作系统任务分为系统任务和用户任务两种。其中，系统任务的优先级 < 50 ，用户任务的优先级 ≥ 50 且 ≤ 255 。优先级大于 255 的为非法任务，应予以剔除。现有一任务序列 `task`（顺序存储），任务数为 n 。每个 `task` 的元素值表示任务的优先级，数值越小，优先级越高。算法 `scheduler` 实现如下功能，将 `task` 中的任务按照系统任务、用户任务依次存放到队列 `system_task` 和 `user_task` 中（队列中元素的值是任务在顺序表 `task` 中对应的下标），并且优先级高的任务排在前面。优先级相同的任务按照在 `task` 中的相对顺序入队排列（即 `task` 序列中在先的任务排在前面）。

顺序表使用 `vector` 或数组，队列使用本学期设计好的顺序队列 ADT

2.3 判断单链表的对称性

设带头结点的单链表的头指针为 `head`，结点结构由 `data` 和 `next` 两个域构成，其中 `data` 域为字符型。在使用单链表 ADT 的基础上，设计一算法判断该链表的前 n 个字符是否中心对称。例如 `x`, `xyx`, `xyyx` 都是中心对称。

参考函数原型：

```
//判断单链表的前 n 个结点是否中心对称
template<class ElemType>
bool Judge_Symmetry( LinkList<ElemType> &L, int num );
```

2.4 AVL 树的判断

目的: 设计并实现一个算法 `Judge_AVL`, 用来判断 1 棵给定的二叉排序树是否是平衡二叉树。

提示:

(1) 若一棵二叉排序树中每个结点的左、右子树的深度之差(平衡因子)的绝对值不超过 1, 则称这样的二叉树为平衡二叉树(AVL 树)。

(2) 在小组任务 12 的基础上, 首先判断该二叉树是否是一棵二叉排序树。如不是, 则可直接给出 `false` (不是 AVL 树); 如是, 进入 (3)

(3) 使用任意的遍历方式对该二叉树进行遍历(测试数据使用非递归的后序遍历得到, 参考个人任务 12, 在 `visit` 函数中计算访问的结点的平衡因子。参照个人任务 13, 但要稍作修改(不要通过数据元素的值, 而是直接通过遍历到的结点地址来定位)。

(4) 如有结点的平衡因子的绝对值超过 1, 则可给出 `false`; 如所有结点的平衡因子的绝对值都不超过 1, 则可给出 `true`。

2.5 判断给定的二叉树是否是二叉排序树

在二叉树的二叉链表存储形式建立的基础上, 使用递归的程序设计方法, 设计并完成判断一棵给定的二叉树是否是二叉排序树的算法。

初始条件: 二叉树 `T`。

操作结果: 若 `T` 是二叉排序树, 则返回 `true`, 否则返回 `false`。

提示:

(1) 若它的左子树不空, 则左子树上所有结点的值均小于它的根结点的值;

(2) 若它的右子树不空, 则右子树上所有结点的值均大于它的根结点的值;

(3) 它的左、右子树也分别为二叉排序树。

(4) 递归遍历, 左孩子的 `key` 比根节点的 `key` 小, 右孩子的 `key` 比根节点的 `key` 大, 一旦有不满足条件的就判定不是。

2.6 希尔排序

对于顺序存储的线性表, 使用 `vector` 或数组, 实现希尔排序算法, 并输出每趟的排序结果。

参考函数原型: (`vector` 版本)

(1) //希尔排序(主函数)

```
template<class ElemType>
```

```
void ShellSort( vector<ElemType> &A, int dlta[], int t ); //t 为希尔排序的趟数
```

(2) //希尔排序(一趟)

```
template<class ElemType>
```

```
void ShellInsert( vector<ElemType> &A, int dk );
```

2.7 堆排序

对于顺序存储的线性表，使用 vector 或数组，实现堆排序算法，并输出每趟的排序结果。

参考函数原型：（vector 版本）

（1）//建堆

```
template<class ElemType>
```

```
void HeapSort( vector<ElemType> &A, int flag ); //flag: 堆类型标记
```

（2）//堆调整（最大堆）

```
template<class ElemType>
```

```
void HeapASCAdjust( vector<ElemType> &A, int hole, int size );
```

（2）//堆调整（最小堆）

```
template<class ElemType>
```

```
void HeapDASCAdjust( vector<ElemType> &A, int hole, int size );
```

2.8 找出从指定结点出发且长度为 m 的所有简单路径

在使用图的邻接表 ADT 的基础上，设计一个算法，按照深度优先搜索的思想找出从指定结点出发且长度为 m 的所有简单路径。并将此算法加入到邻接表 ADT 中，在邻接表 ADT 中提供一个公有的成员函数 FindPath(start, m)。

提示：

（1）这个问题相当于从指定结点开始深度优先遍历，而且遍历的深度正好为 m。为此，在遍历时需要记住遍历的深度，当深度达到 m 时，就不需要递归了。此时需要输出这条路径，因此在遍历的过程中还需要记住整条路径。

（2）由于深度优先遍历是用递归实现的，所以 FindPath 函数最好也设计两个。一个是共有的 FindPath 函数，供用户使用（外壳）；另一个是私有的 FindPath 函数，实现递归的遍历。公有的 FindPath 函数调用私有的 FindPath 函数找出这些路径。

（3）在调用递归的 FindPath 函数前，外壳函数还需要做一些辅助工作：

1) 要找的是长度为 m 的简单路径，因此路径上不能有相同的结点，于是定义了一个数组 visited 记录结点是否在路径上。

2) 当路径长度等于 m 时要输出这条路径，于是定义了一个数组 stack 保存这条路径。每访问一个结点，都要把结点记录在 stack 中。

3) 递归的 FindPath 函数有 6 个参数。第 1 个参数是遍历的起点的序号；第 2 个参数是要求的路径长度；第 3 个参数是符合要求的路径数目；第 4 个参数是当前路径中的结点数，当前路径的长度是结点数减 1；第 5 个参数是 visited 数组，记录结点是否在路径上；第 6 个参数是一个用于记录路径上结点序号的数组，作用和栈类似。

4) 调用 FindPath 函数时要指出从哪一个结点出发，而递归的 FindPath 函数的参数是起点的序号。递归的 FindPath 函数首先将起点放入这条路径，并标记这个结点已被访问，然后判断路径长度是否是 m。如果长度已达到 m，则输出这条路径，并将最后一个结点从路径上删除，返回上一层调用，检查是否还有其它的途径；否则逐个检查起点的后继结点。如果该后继结点没有被访问过，则对该结点调用递归的 FindPath 函数继续寻找。在所有的后继都检查后，表示这条路径处理完毕。将起始结点从这条路径上删除，返回上一层调用。

2.9 获取 AOE 网的关键路径

建立一个有向网 AOE 网，设计并完成一算法 `Get_CriticalPath()`，获取关键路径。该路径仅输出，不须保存。

提示：在 AOE 网中，

- (1) 关键活动：开始时间余量为 0 的活动。即活动的最早开始时间等于它的最迟开始时间。
- (2) 根据各个顶点的 V_e 和 V_l 值，在求得每条弧 s 的最早开始时间 $e[s]$ 和最迟开始时间 $l[s]$ 后，若某条弧满足条件 $e[s]=l[s]$ ，该弧所对应的活动即为关键活动。
- (3) 关键路径：由关键活动所形成的从源点到汇点的每一条路径（注意：关键路径可能有多条）。

2.10 油管铺设

八口海上油井相互间距离如下表所示（样例 1 数据）。其中 1 号井离海边距离最近，为 5.6 海里。现计划从海岸铺设油管将各油井连接起来，如何设计才能使得油管的长度最短。（为便于检修，油管只允许在油井处分叉）。

从~到	2	3	4	5	6	7	8
1	1.3	2.1	0.9	0.7	1.8	2.0	1.8
2		0.9	1.8	1.2	2.8	2.3	1.1
3			2.6	1.7	2.5	1.9	1.0
4				0.7	1.6	1.5	0.9
5					0.9	1.1	0.8
6						0.6	1.0
7							0.5

提示：

基于最小生成树思想解决实际问题。

2.11 快速排序的应用

已知线性表 $(a_1, a_2, a_3, \dots, a_n)$ 按顺序存于内存，每个元素都是非零整数。在使用顺序表 ADT 的基础上，试设计基于快速排序的思想把所有值为负数的元素移到全部正数值元素前边的算法：例： $(x, -x, -x, x, x, -x, x)$ 变为 $(-x, -x, -x, x, x, x, x)$ 。

提示：要求重排 n 个元素且以顺序存储结构存储的线性表，使得所有值为负数的元素移到正数元素的前面。这可采用快速排序的思想来实现。只是比较的标准是元素是否为负数。因此枢轴元素的值为 0 (不是线性表中的元素)。基本思路是，先设置好上、下界和枢轴值 (0)，

然后执行一趟快速排序，即利用震荡交替法分别从线性表的两端查找正数和负数，找到后互相交换，直到上下界相遇。

参考函数原型：

```
template<class ElemType>
void Rearrange( SqList<ElemType> &A );
```

2.12 仓储货物重排

在智能仓储系统中，初始状态下货物按矩阵 source 排列，而订单需求要求货物按矩阵 target 重新排列。机械臂可以：

- (1) **移动货物位置**（任意重排）；
- (2) **替换货物类型**（将某位置的货物替换为另一种类型）。

由于替换货物的操作比移动货物更耗时且成本更高，要求设计一个算法，在满足订单需求的前提下，**最小化货物替换次数**。

示例 1：

输入：

```
2 2 // 表示矩阵为 2 行 2 列
1 3 // 初始：A 区放货物 1，B 区放货物 3
5 4 // C 区放货物 5，D 区放货物 4
3 1 // 目标：A 区需放货物 3，B 区需放货物 1
6 5 // C 区需放货物 6，D 区需放货物 5
```

输出：1

解释：

第一行输入 2 2 表示方阵为 2 行 2 列，后面跟着的两行为 source，再两行为 target。

最佳方案为：将 D 区的货物 4 替换为 6，其余通过移动完成。

移动路径：A→B，B→A，C→D，D→C（替换为 6）。

因此从 source 到 target 所需要的最少货物替换次数为 1。

示例 2：

输入：

```
3 3
0 0 0
0 0 0
0 0 0
1 2 3
4 5 6
7 8 9
```

输出：0

解释：两个矩阵的元素完全相同，只是排列顺序不同，无须替换货物。

2.13 智能物流中心任务调度

随着电子商务和智慧物流的发展，现代物流中心每天需要处理数以万计的包裹。

一个包裹从进入物流中心到最终送达客户手中，需要经过多个处理环节。物流中心的任务编号及名称如下表所示：

任务编号	任务名称
1	货物入库
2	自动分拣
3	安全检查
4	装车运输
5	城市配送
6	客户签收
7	异常件处理
8	配送结果回传
9	客户评价
10	订单归档

这些任务之间的依赖关系如下：

- 货物必须先入库，才能进行自动分拣和安全检查；
- 自动分拣和安全检查可以并行进行，但两项任务都完成后才能开始装车运输；
- 装车运输完成后，才能进行城市配送；
- 城市配送完成后，客户才能签收系统将根据配送结果进入后续处理阶段；
- 对于正常配送的订单，客户签收后才能确认配送完成；
- 对于配送过程中出现异常的订单，需要先进行异常件处理；
- 客户签收和异常件处理完成后，系统才能生成并回传最终配送结果；
- 配送结果回传后，客户才能对本次物流服务进行评价；
- 客户评价完成后，订单才能进入归档状态并结束整个物流流程。

由于各个环节之间存在严格的先后依赖关系，因此物流调度必须保证：

- (1) 前置任务完成后，后续任务才能开始；
- (2) 不允许出现违反依赖关系的调度方案；
- (3) 当多个任务同时满足执行条件时，应优先执行编号较小的任务；
- (4) 如果任务依赖关系存在冲突，则无法生成配送计划。

请你编写一个物流任务调度程序，为物流中心生成唯一的合法执行顺序。

问题描述：

物流中心共有 n 个任务。任务编号为 $1 \sim n$ 。

任务间的依赖关系为 $u \rightarrow v$ 。表示：任务 u 必须先完成，任务 v 才能开始执行。即 $u \rightarrow v$ 。

请输出：

- (1) 唯一合法的任务执行顺序；
- (2) 若无法完成调度，则输出：false

注意：

当多个任务同时满足执行条件时，优先执行编号较小的任务。

输入格式：

第一行 2 个整数：n m （n：任务总数，m：依赖关系数量）

接下来 m 行，每行 2 个整数：u v （表示 $u \rightarrow v$ ）

输出格式：

若存在合法调度方案，输出一行调度方案：任务执行顺序（任务编号之间以 $\rightarrow$ 分隔）。

若存在不合法调度方案，输出：false

数据范围：

$1 \leq n \leq 100$

$0 \leq m \leq 500$

2.14 大数据分析平台热点数据缓存

某大型电商企业建设了一套大数据分析平台，用于存储和分析海量业务数据，包括用户行为日志、订单数据、商品信息、物流数据和营销数据等。

在实际使用过程中，数据分析师会频繁访问某些热门数据集。例如：

- 用户画像数据集
- 商品销售统计数据集
- 订单流水数据集
- 物流配送数据集
- 用户评价数据集

由于这些数据集存储在分布式数据库中，每次读取都会消耗较长时间。为了提高查询效率，系统需要设计一套**热点数据缓存机制（Hot Data Cache）**：缓存容量有限，只能保存部分最近访问的数据集。当缓存空间已满时，系统需要自动淘汰长时间未被访问的数据集，从而为新的热点数据腾出空间。

该机制采用如下策略：当缓存已满且需要加载新的数据集时，优先淘汰最近最少使用（**Least Recently Used, LRU**）的数据集。

请你设计并实现该热点缓存机制。

设计一个容量固定的数据缓存系统，支持以下两种操作：

(1) 加载数据集

`put(datasetID, accessCount)`

表示将编号为 datasetID 的数据集加载到缓存中，其访问次数为 accessCount。

具体规则如下：

- 如果该数据集已经存在于缓存中，则更新其访问次数；
- 更新后，该数据集视为最近被访问；
- 如果该数据集不存在，则将其加入缓存；
- 如果缓存已满，则必须先删除最近最少使用的数据集，再插入新数据集；

- 新插入的数据集视为最近被访问。

(2) 查询数据集

`get(datasetID)`

表示访问指定数据集。

具体规则如下：

- 如果该数据集存在于缓存中，则输出其访问次数；
- 如果不存在，则输出 -1；
- 成功访问的数据集将被视为最近访问的数据集。

最近最少使用（LRU）规则说明：**缓存按照最近访问时间维护顺序。**

例如：

当前缓存容量为：3

缓存内容（从最近访问到最久未访问）：103 102 101

若此时执行：`get 102`

则 102 成为最近访问的数据集：102 103 101

若再执行：`put 104 500`

由于缓存已满，需要淘汰最久未访问的数据集：101

最终缓存状态：104 102 103

输入格式：

第一行输入一个整数：`capacity`（表示缓存容量）

第二行输入一个整数：`n`（表示操作次数）

接下来 `n` 行，每行表示一次操作。

输出格式：

对于每一个 `get` 操作：

输出一行结果。

若数据集存在：输出其访问次数

否则：输出 -1

数据范围：

$1 \leq \text{capacity} \leq 100000$

$1 \leq n \leq 100000$

$1 \leq \text{datasetID} \leq 10^9$

$0 \leq \text{accessCount} \leq 10^9$

函数 `get` 和 `put` 须以 $O(1)$ 的平均时间复杂度运行，否则可能无法通过所有测试数据。

2.15 智能仓储分拣机器人调度

某大型电商企业建设了一座智能物流仓库。仓库中部署了一台自动分拣机器人，该机器人负责完成订单包裹的分拣工作。

每天会有大量订单陆续到达仓库，不同订单的处理时间不同：

- 小件商品处理时间较短；
- 大件商品处理时间较长；
- 复杂订单需要更多时间完成分拣。

由于仓库只有一台分拣机器人，因此同一时刻只能处理一个订单。

为了提高整体分拣效率，仓库采用如下调度策略：

规则 1：若机器人空闲且没有可处理订单，机器人保持等待状态，直到有新订单到达。

规则 2：若机器人空闲且等待队列非空，优先选择处理时间最短的订单。

规则 3：若多个订单处理时间相同，选择订单编号最小的订单。

规则 4：订单开始处理后，不能被中断，必须执行完成。

规则 5：机器人完成当前订单后立即选择下一个订单。

设仓库共有 n 个订单。订单编号为 $0 \sim n-1$ 。

对于第 i 个订单，有两个参数：arrivalTime、processingTime
表示：

(1) arrivalTime：订单进入仓库等待队列的时间（格式为 HH:MM）

(2) processingTime：机器人完成该订单所需时间（整数，秒）

请你根据订单到达时间和处理时间，设计一个算法，输出机器人完成订单的处理顺序。

注意：假设该智能仓储中心每天 8:00 开始工作，订单会在一天中的不同时刻到达仓库。你的算法应当将订单到达时间统一转换为“自当天 00:00 开始经过的分钟数”进行存储。

例如：08:30 $\rightarrow$ 510；09:15 $\rightarrow$ 555；14:20 $\rightarrow$ 860

输入格式：

第一行输入整数： n （表示订单数量）

接下来 n 行：arrivalTime processingTime（表示一个订单的信息。其中，第 i 行对应订单 i ）

到达时间范围： $00:00 \leq \text{arrivalTime} \leq 23:59$

订单数量： $1 \leq n \leq 100000$

输出格式：

输出一行：订单处理顺序（订单编号之间用“->”分隔）

3. 小组任务

3.1. 任务背景

近年来,随着智慧交通和智慧城市建设的不断推进,城市轨道交通信息化水平不断提高。地铁路径规划系统已经成为城市轨道交通信息服务的重要组成部分。通过路径规划算法,乘客可以快速获得最佳出行方案;通过运营管理系统,管理人员可以及时掌握网络运行状态,提高地铁运营效率。

上海地铁目前拥有多条运营线路和数百个运营站点,形成了规模庞大且结构复杂的轨道交通网络。面对如此复杂的网络结构,传统人工方式已经难以满足路径查询与运营管理需求,因此需要利用数据结构与算法技术建立高效的路径规划与管理系统。

本课程设计小组任务以真实上海地铁网络为研究对象,采用图结构对地铁网络进行建模。系统将地铁站点抽象为图中的顶点,将站点之间的连接关系抽象为图中的边,通过最短路径算法、广度优先搜索算法等数据结构课程核心知识,实现路径规划与运营管理功能。

在基础路径规划功能的基础上,系统进一步扩展实现:

- (1) 站点关闭管理;
- (2) 换乘站整体关闭管理;
- (3) 线路停运管理;
- (4) 全网停运与恢复;
- (5) 受影响区域分析;
- (6) 网络连通性分析;
- (7) K 条最优路径查询等功能。

通过这些功能的实现,系统能够较好地模拟真实地铁运营场景,提高课程设计的工程实践价值。

3.2. 基本功能

系统采用 C++ 语言开发,利用邻接表图结构实现地铁网络建模,主要包含以下功能:

1) 地铁数据的加载与建图

- 从 CSV 文件加载地铁站点与线路信息;
- 构建邻接表图结构。

2) 支持多种路径规划算法

- 最短时间路径(最少耗时)查询;
- 最少换乘路径查询;
- K 条最优路径查询(例如时间最短的前 3 条路径)。

3) 支持运营管理及网络分析

- 可通过 CSV 批量修改站点开放状态(开放/关闭);
- 可手工修改站点开放状态(开放/关闭);
- 可恢复站点初始状态(全开放);
- 显示当前关闭的站点名单。

- 关闭站点受影响区域分析。

4) 系统交互

- 控制台二级菜单交互
- 路径可视化输出（指定线路的站点信息、换乘信息）；
- 路径规划信息需包括路径总时间、换乘站点、换乘次数、实际经过站点数。

5) 进阶功能（可选加分）

- 用户起点/终点模糊搜索（如部分输入“莘”匹配“莘庄”）；
- 可视化地铁网络图（图形界面）

3.3. 主要功能模块

1) 数据加载与建模模块

- 读取 Station.csv 和 Edge.csv；
- 构建 stations 映射与 adj 邻接表。

2) 最优路径规划模块

- shortestPath: 规划单条最短时间路径（时间相同时，按换乘次数升序）。
- kShortestTimePath: 规划前 3 条最短时间路径（时间相同时，按换乘次数升序）。
- minTransferPath: 规划单条换乘次数最少路径（换乘次数相同时，按时间升序）。
- kShortestTimePaths: 规划前 3 条换乘次数最少路径（换乘次数相同时，按时间升序）。
- 剪枝策略：避免回路、重复换乘、禁止经过关闭站点等。

3) 路径输出与去重模块

- 显示路径站点、时间、换乘次数、实际经过站数（不含换乘）；
- 判断路径重复：通过线路序列或站点序列去重。

4) 线路站点信息/运营状态管理模块

- updateStationCsvStatus: 基于站点关闭列表数据文件（update_station_status.csv）批量更新站点数据文件 Station.csv。
- resetStationStatus: 基于站点初始化数据文件 Station_init.csv 恢复 Station.csv 的初始状态。
- showClosedStations: 显示当前关闭站点名单。
- showInfobyLine: 显示指定地铁线路相关站点信息。

5) 用户交互模块

- 二级菜单实现交互，可选择路径查询、站点状态修改、站点状态查看、恢复初始状态等功能，适用于多轮交互式测试与验证。

```
==== 地铁路径规划系统 ====
1. 线路站点信息/运营状态管理
2. 所需时间最短路径规划
3. 所需换乘次数最少路径规划
4. 退出系统
请输入选项编号：
```

图 1 主菜单（一级）

```
-- 线路站点信息/运营状态管理 --
1. 从 CSV 文件批量更新站点开启/关闭状态
2. 手工更新站点开启/关闭状态
3. 显示当前关闭站点
4. 恢复所有站点初始状态
5. 显示线路站点信息
6. 受关闭站点影响站点分析
7. 站点查询
8. 返回上级菜单
请输入选项编号：
```

图 2 站点状态管理子菜单（二级）

-- 所需时间最短路径规划 --
1. 单条所需时间最短路径
2. 3条所需时间最短路径
3. 返回上级菜单
请输入选项编号：

图 3 时间最优子菜单（二级）

-- 所需换乘次数最少路径规划 --
1. 单条换乘次数最少路径
2. 3条换乘次数最少路径
3. 返回主菜单
请输入选项编号：

图 4 换乘最优子菜单（二级）

3.4. CSV 数据文件说明

1) Station.csv（含站点开放/关闭状态）

- 记录所有地铁站点信息，包括站点 ID、名称、所属线路、开放状态（开放/关闭）。系统运行过程中，路径搜索功能能够根据该文件中的状态字段判断站点是否可通行，支持动态修改以模拟线路维护或突发事件。（站点开放状态默认为“开启”）。

- 需包含上海地铁 1 号线至 18 号线、浦江线、市域机场线所有站点信息。

- 包括站名相同的换乘站点在内，共计 525 个站点。

- 参考上海地铁官网：<https://service.shmetro.com/axlcz/index.htm>

2) update_station_status.csv（批量更新站点状态）

- 读取该文件后能够自动更新 Station.csv 中对应站点的状态，实现快速状态管理。

3) Edge.csv（地铁线路数据）

- 描述地铁站点之间的连接关系和通行耗时。其每一行代表一条边（即一段地铁线路）。

- 包含以下字段：

起点站 ID：邻接开始站点的 ID（与 Station.csv 中的 ID 一致）。

终点站 ID：邻接结束站点的 ID。

线路名称：该边所对应的地铁线路名称（如“1 号线”、“2 号线”、“换乘”等），也可用于判断是否换乘。

运行方向：内外圈信息（仅针对 4 号线，“内圈”、“外圈”）

通行时间：相邻站点行驶时间。

- 本文件用于构建邻接表图结构，是路径规划算法的基础数据来源。

- 包括换乘边在内，总共 1226 条边。**注意：**由于 A、B 相邻站点实际运行时，A->B 与 B->A 的行驶时间有可能不相同，最好按有向边处理，设置为 2 条边。

- 换乘时间统一按 5 分钟计算。

- 相邻站点间行驶所需时间参考上海地铁官网。

起点站ID	终点站ID	所属线路	运行方向	通行时间
1001	1002	1号线		2
1002	1001	1号线		2
1001	1422	换乘		5
1422	1001	换乘		5
1002	1003	1号线		2
1003	1002	1号线		2
1003	1004	1号线		2
1004	1003	1号线		2
1004	1005	1号线		3
1005	1004	1号线		3
1005	1408	换乘		5
1408	1005	换乘		5
1005	1487	换乘		5
1487	1005	换乘		5
1005	1006	1号线		2
1006	1005	1号线		2
1006	1453	换乘		5
1453	1006	换乘		5
1006	1007	1号线		2
1007	1006	1号线		2
1007	1410	换乘		5
1410	1007	换乘		5

站点ID	站点名	所属线路	运营状态
1001	莘庄	1号线	开启
1002	外环路	1号线	开启
1003	莲花路	1号线	开启
1004	锦江乐园	1号线	开启
1005	上海南站	1号线	开启
1006	漕宝路	1号线	开启
1007	上海体育馆	1号线	开启
1008	徐家汇	1号线	开启
1009	衡山路	1号线	开启
1010	常熟路	1号线	开启
1011	陕西南路	1号线	开启
1012	一大会址·黄陂	1号线	开启
1013	人民广场	1号线	开启
1014	新闸路	1号线	开启
1015	汉中路	1号线	开启
1016	上海火车站	1号线	开启
1017	中山北路	1号线	开启
1018	延长路	1号线	开启
1019	上海马戏城	1号线	开启
1020	汶水路	1号线	开启
1021	彭浦新村	1号线	开启
1022	共康路	1号线	开启
1023	通河新村	1号线	开启
1024	呼兰路	1号线	开启
1025	共富新村	1号线	开启
1026	宝安公路	1号线	开启
1027	友谊西路	1号线	开启
1028	富锦路	1号线	开启

图 5 Station.csv 截图

图 6 Edge.csv.csv 截图

站点名	线路	运营状态
漕宝路	1号线	关闭
陆家嘴	2号线	关闭
萧塘	5号线	关闭
萧塘	7号线	开启

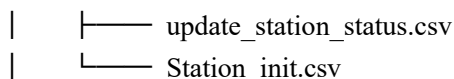
图 7 update_station_status.csv 截图

3.5. 部分项目代码结构（建议）

```

/project-root
|
├── main.cpp           //主程序入口
├── station.h / .cpp   //从 Station.csv 文件中读取数据，实现站点信息的加载与管理
├── graph.h / .cpp     //从 Edge.csv 文件中读取数据，构建图的邻接表结构
├── menu.h / .cpp      //二级菜单交互
├── pathfinder.h / .cpp //实现时间最短路径、最少换乘路径等算法
├── data/              //存放相关数据文件
|   ├── Station.csv
|   └── Edge.csv

```



3.6. 小组成员分工建议（4 人为例）

成员	分工模块	技术点
成员 A	地铁图建模模块	邻接表、结构体定义、CSV 解析
成员 B	最短时间/最少换乘路径算法实现	优先队列、Dijkstra、剪枝
成员 C	路径输出与候选路径逻辑	多路径生成、排序、路径去重
成员 D	动态站点管理 + 菜单交互	CSV 更新、状态过滤、菜单设计

3.7. 实验报告的要求

1) 交互过程或设计结果

- 使用 AI 编程助手的学生，需提交完整的交互过程记录，包括提问、AI 回复和个人思考。将根据提问的技巧和思考的深度进行评分。
- 不使用编程助手的学生，需提交设计思路文档。例如：使用什么关键词在百度上搜索，得到什么网页，经过思考，得到什么结论，再使用什么关键词搜索...，得到了什么设计结果或者代码等。将根据设计的合理性、创新性和完整性进行评分。

2) 测试及结果

- 对其中的功能进行完善的测试，基本上，每个功能提交至少 5 组不同的测试数据及其运行结果。测试应该包括单个功能的测试以及多个功能的关联测试，例如封闭一个换乘车站后，对路线查询结果的影响等。
- 测试数据的复杂性和完备性将影响得分。

4) 实验报告内容

- 实验报告撰写格式规范、结构完整，包括设计、实现、使用示例、测试等内容。
- 需要介绍清楚：测试数据的描述、对测试数据的分析以及据此进行进一步的与 AI 编程助手的交互和调试等。测试数据最好用表格呈现，包括测试目的、输入、期望输出、实际输出、成败结果等。
- 整体设计（包括类及函数的设计和调用关系等）、一些稍微复杂一些的算法流程图等，需要用图展示。

注意：

- 程序应具备良好的可读性和可维护性。
- 程序的运行效率也将作为评分的参考因素。
- 提交的作业应包括完整的源代码和实验报告。
- 源代码应有良好的风格和注释。

3.8. 运行效果图（部分）

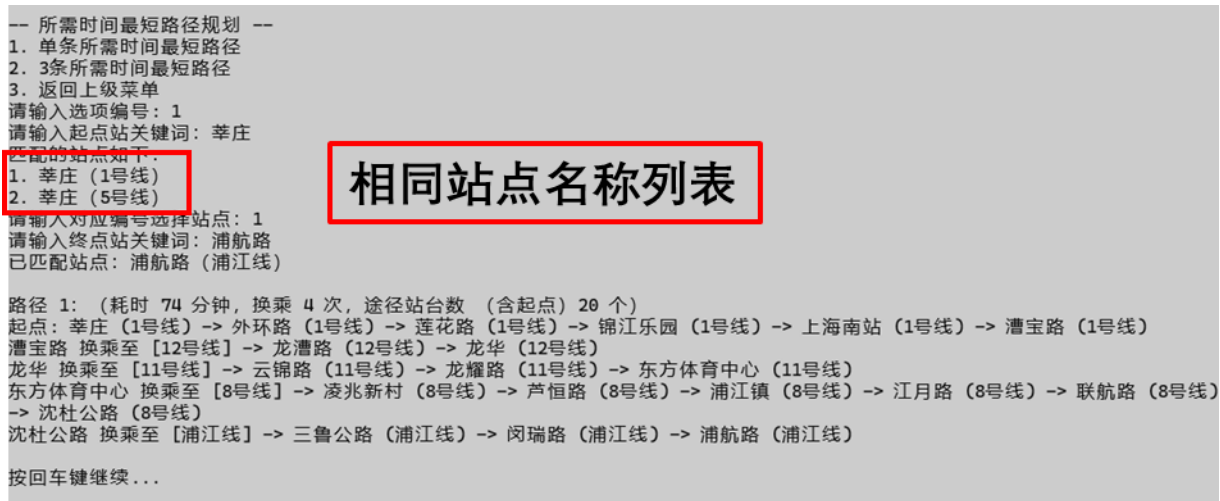


图 8 单条所需时间最短路径效果图

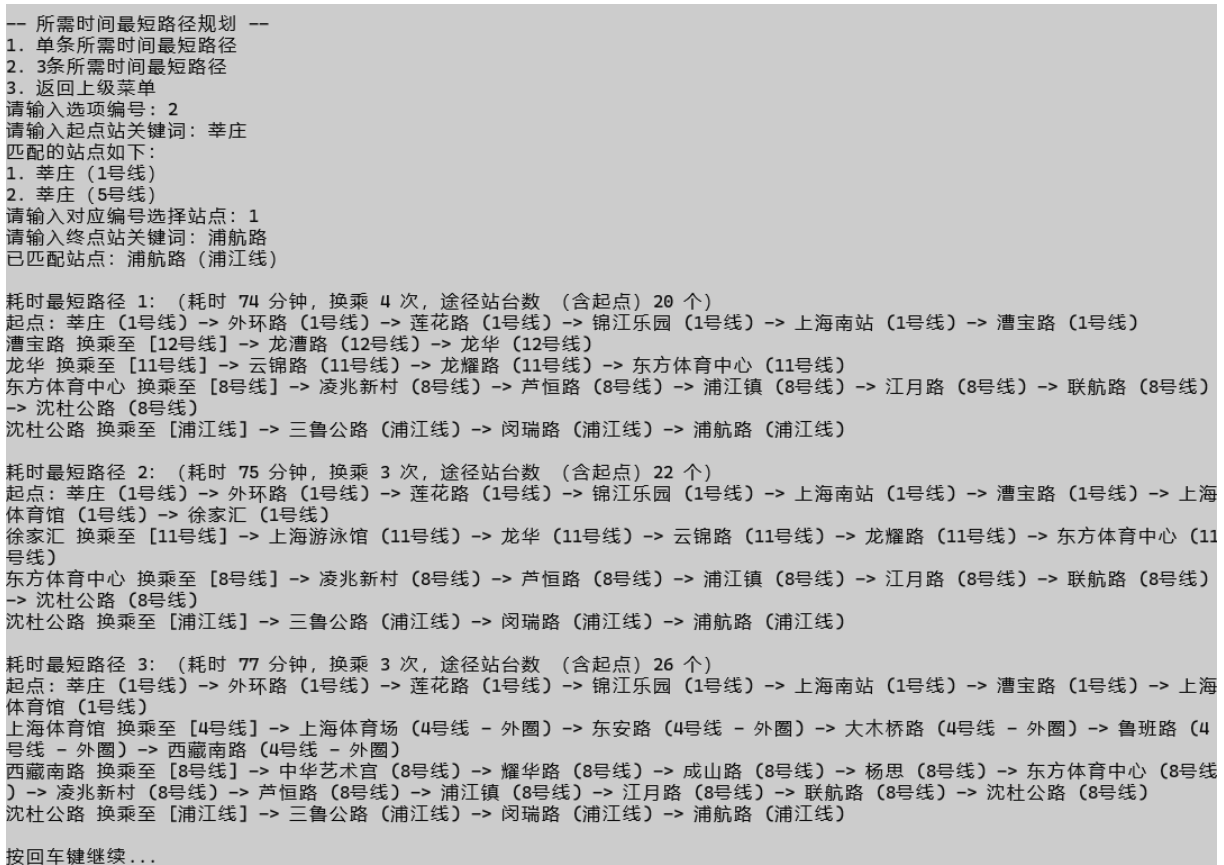


图 9 3 条所需时间最短路径效果图

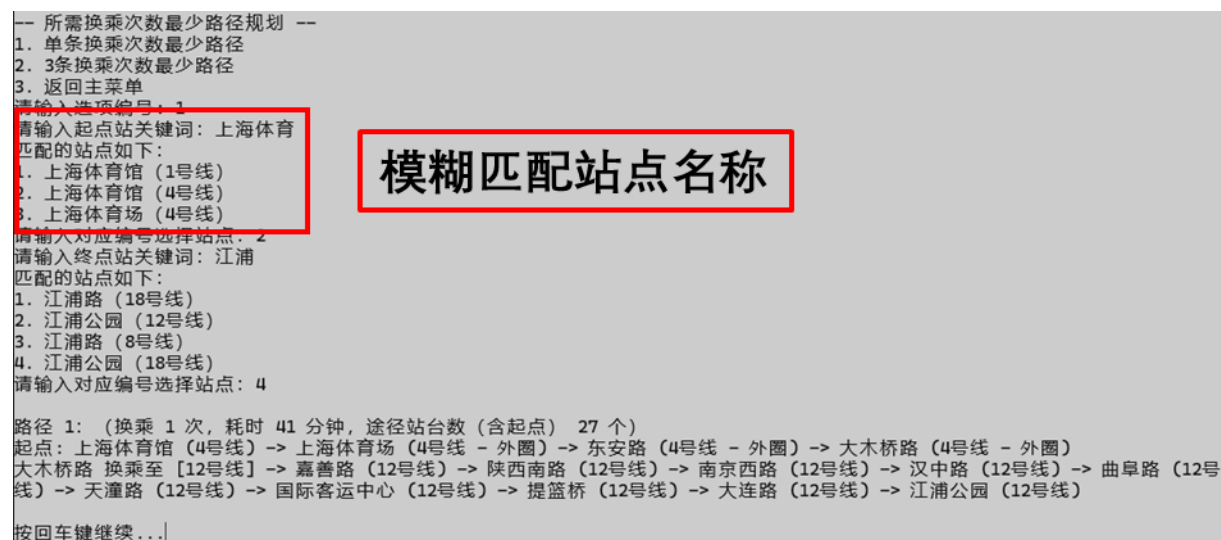


图 10 单条换乘次数最少路径效果图

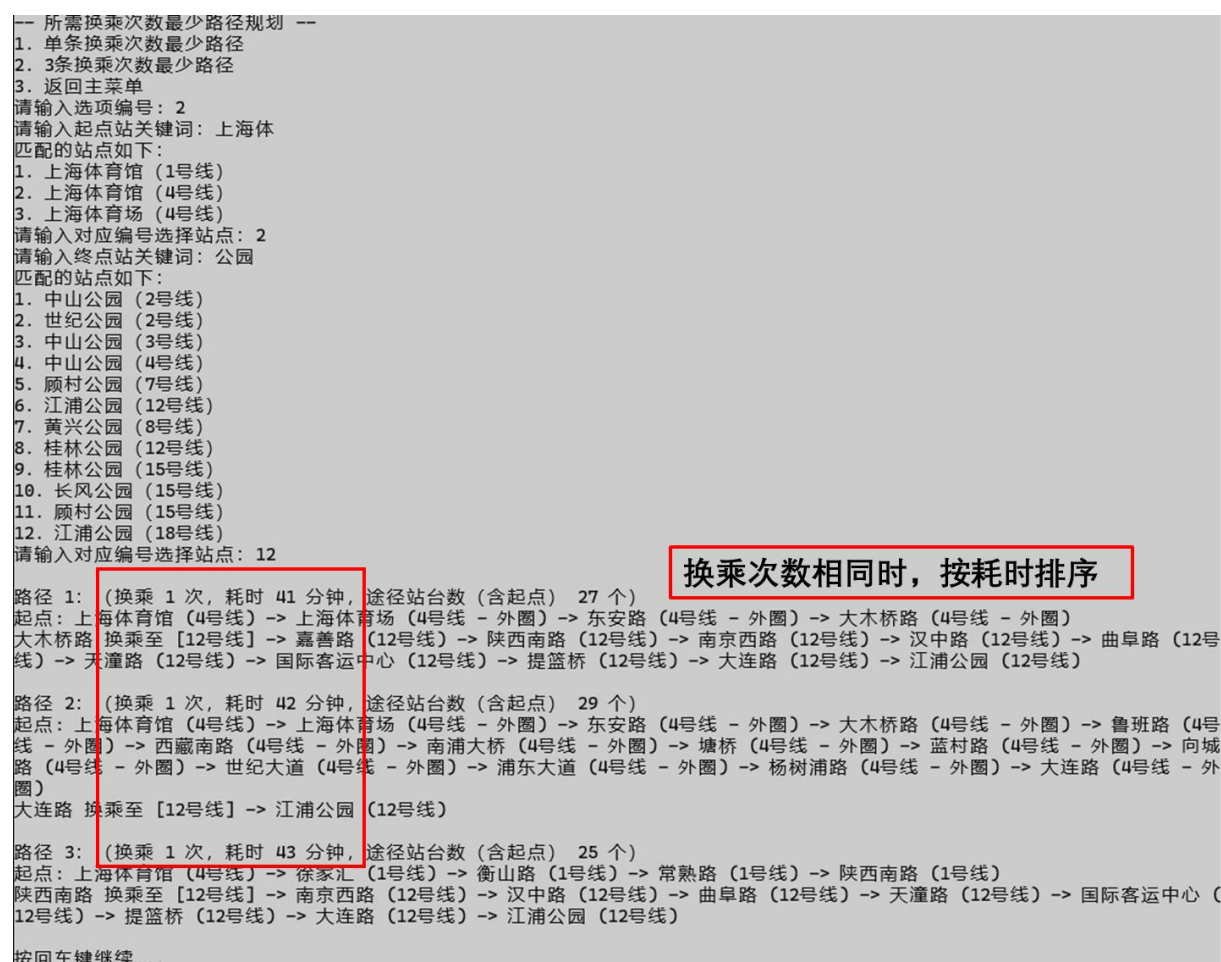


图 11 3 条换乘次数最少路径效果图

```
-- 线路站点信息/运营状态管理 --
1. 从 CSV 文件批量更新站点开启/关闭状态
2. 手工更新站点开启/关闭状态
3. 显示当前关闭站点
4. 恢复所有站点初始状态
5. 显示线路站点信息
6. 返回上级菜单
请输入选项编号：3

当前关闭的站点列表：
关闭站点1：漕宝路，1号线
关闭站点2：萧塘，5号线
```

图 12 显示当前关闭站点效果图

```
-- 所需时间最短路径规划 --
1. 单条所需时间最短路径
2. 3条所需时间最短路径
3. 返回上级菜单
请输入选项编号：1
请输入起点站关键词：莘庄
匹配的站点如下：
1. 莘庄（1号线）
2. 莘庄（5号线）
请输入对应编号选择站点：1
请输入终点站关键词：浦航路
已匹配站点：浦航路（浦江线）
```

漕宝路（一号线）站点关闭，相关路径被规避

```
耗时最短路径 1:（耗时 79 分钟，换乘 5 次，途径站台数（含起点）29 个）
起点：莘庄（1号线）-> 外环路（1号线）-> 莲花路（1号线）-> 锦江乐园（1号线）-> 上海南站（1号线）
上海南站 换乘至【3号线】-> 石龙路（3号线）-> 龙漕路（3号线）
龙漕路 换乘至【12号线】-> 龙华（12号线）
龙华 换乘至【11号线】-> 云锦路（11号线）-> 龙耀路（11号线）-> 东方体育中心（11号线）
东方体育中心 换乘至【8号线】-> 凌兆新村（8号线）-> 芦恒路（8号线）-> 浦江镇（8号线）-> 江月路（8号线）-> 联航路（8号线）
-> 沈杜公路（8号线）
沈杜公路 换乘至【浦江线】-> 三鲁公路（浦江线）-> 闵瑞路（浦江线）-> 浦航路（浦江线）
按回车键继续...
```

图 13 线路规划规避关闭站点效果图

```

-- 线路站点信息/运营状态管理 --
1. 从 CSV 文件批量更新站点开启/关闭状态
2. 手工更新站点开启/关闭状态
3. 显示当前关闭站点
4. 恢复所有站点初始状态
5. 显示线路站点信息
6. 返回上级菜单
请输入选项编号：5
线路编号列表 (1 - 20)
1 - 1号线      2 - 2号线
3 - 3号线      4 - 4号线
5 - 5号线      6 - 6号线
7 - 7号线      8 - 8号线
9 - 9号线      10 - 10号线
11 - 11号线    12 - 12号线
13 - 13号线    14 - 14号线
15 - 15号线    16 - 16号线
17 - 17号线    18 - 18号线
19 - 浦江线    20 - 市域机场线

请输入要查询的线路编号 (1 - 20)：17

```

图 14 线路站点信息查询（线路指定）效果图

```

请输入要查询的线路编号 (1 - 20)：17
17号线共有14个站点
○ 虹桥火车站 ( 换乘：2号线)
|
○ 国家会展中心
|
○ 蟠龙路
|
○ 徐盈路
|
○ 徐泾北城
|
○ 嘉松中路
|
○ 赵巷
|
○ 汇金路
|
○ 青浦新城
|
○ 漕盈路
|
○ 淀山湖大道
|
○ 朱家角
|
○ 东方绿舟
|
○ 西岑

```

图 15 指定线路站点信息查询效果图

```

-- 线路站点信息/运营状态管理 --
1. 从 CSV 文件批量更新站点开启/关闭状态
2. 手工更新站点开启/关闭状态
3. 显示当前关闭站点
4. 恢复所有站点初始状态
5. 显示线路站点信息
6. 返回上级菜单
请输入选项编号：2
请输入待修改站点关键词 (exit退出)：莘庄
匹配的站点如下：
1. 莘庄 (1号线)
2. 莘庄 (5号线)
请输入对应编号选择站点：1
莘庄,1号线,开启
请输入站点状态 (开启/关闭) > 开启

```

图 16 手工更新站点开启/关闭效果图

```
-- 线路站点信息/运营状态管理 --
1. 从 CSV 文件批量更新站点开启/关闭状态
2. 手工更新站点开启/关闭状态
3. 显示当前关闭站点
4. 恢复所有站点初始状态
5. 显示线路站点信息
6. 受关闭站点影响站点分析
7. 返回上级菜单
请输入选项编号：6
请输入站点名称：莘庄
匹配的站点如下：
1. 莘庄（1号线）
2. 莘庄（5号线）
请输入对应编号选择站点：1

=====
关闭站点：莘庄(1号线)

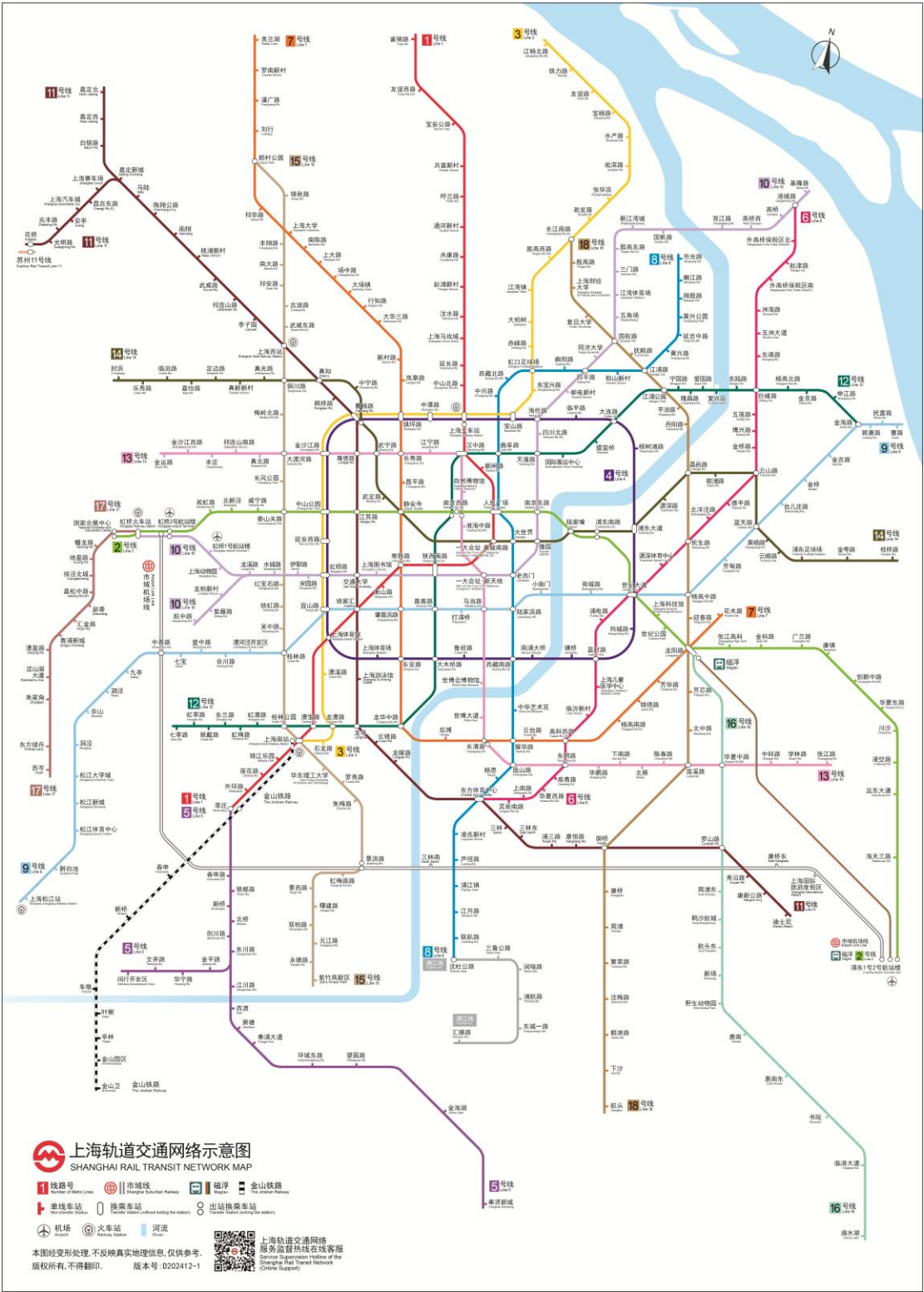
受影响线路：
1号线

直接受影响站点：
外环路(1号线)
外环路(1号线)
无相邻站点

影响等级：中
=====
```

图 17 受关闭站点影响站点分析效果图

附录 A 上海地铁网络示意图



附录 B 小组任务涉及到的知识点

序号	知识点	应用对象
1	结构体 (Struct)	Station、Edge、Path 等数据对象设计
2	顺序表 (Vector)	站点表、边表、路径存储
3	字符串 (String)	站名、线路名处理
4	文件操作 (CSV)	Station.csv、Edge.csv 等读写
5	哈希表 (unordered_map)	站点名称与 ID 映射
6	图 (Graph)	地铁网络建模
7	邻接表 (Adjacency List)	地铁线路存储
8	队列 (Queue)	BFS 搜索
9	优先队列 (Priority Queue)	Dijkstra、Yen 算法
10	栈 (Stack)	路径恢复与输出
11	集合 (Set)	路径去重
12	深度优先搜索 (DFS)	网络连通性分析
13	广度优先搜索 (BFS)	最少换乘路径、影响区域分析
14	Dijkstra 最短路径算法	时间最优路径
15	Yen K 短路径算法	K 条最优路径
16	连通分量分析	网络断裂检测
17	图遍历算法	网络分析与状态检测
18	查找算法	站点查询、线路查询
19	状态维护机制	站点关闭/恢复
20	剪枝思想	K 短路径、候选路径生成